HYTech LMS — Developer Turnover Brief

System: HYTech Learning Management System · **Organization:** HYT Global Institute
Application root: `HYTech/` · **Region:** `asia-southeast1`
Verified against the repository: 30 July 2026 · **Classification:** Internal

This is the short version — everything you need to take ownership, in reading order. The exhaustive reference is HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.md; this brief does not replace it for one-time tasks like standing up a brand-new Firebase environment. Where the two disagree, the long guide wins on detail and this one wins on currency.

> **Your first hour:** read §1–§3, run §4 against **staging**, then read §7 and §8. Do not touch production until you have read §12 and §14.

---

1. What this is

A React single-page LMS on Firebase, serving three roles:

Role	Owns
Administrator	Users, sectors, course templates, classes, settings, logs, ID requests, incidents
Trainer	Class delivery, trainees, co-trainers, content, assessments, grading, graduation
Student / trainee	Registration, joining classes, content, assessments, submissions, progress

Most reads and some writes go **straight from the browser to Firebase** under Security Rules. Anything that must be authoritative, privileged, recursive, or race-safe goes through a callable Cloud Function.

```
Browser — React SPA (Vite build, served by Firebase Hosting)
   ├── Firebase Authentication ..... identity only
   ├── Cloud Firestore ............. app data + realtime listeners
   ├── Cloud Storage ............... avatars, branding, class files, submissions
   ├── App Check ................... reCAPTCHA v3 attestation
   └── Cloud Functions ............. trusted grading, progress, graduation, admin
```

There is no Redux and no central store. State lives in component state, `AuthContext`, `ToastContext`, custom hooks, and Firestore `onSnapshot` listeners. `src/utils/firestoreService.js` is the only data-access layer.

---

2. Environments — and the one trap that will bite you

Environment	Firebase project	Use
Production	`hyt-global-institute-lms`	Live service, real learner records
Staging	`hytech-lms-staging`	Integration, role QA, release validation
Emulator	`demo-hytech-lms`	Local Firestore/Storage rules tests

> ⚠️ `.firebaserc` **sets** `default` **to PRODUCTION.** A bare `firebase deploy` or `firebase firestore:delete` hits the live project. Always pass `--project staging` explicitly, or use the scoped npm scripts.

Two independent things can point at different projects: the **build** (`.env.*.local`) and the **CLI target** (`--project`). Check both. Confirm with `firebase use` and `firebase projects:list` before every deploy.

Never use production data or credentials for local testing.

3. Stack baseline

Area	Baseline
UI	React 18, React Router 7.18.2, Tailwind CSS 3, Lucide React
Build	Vite 8 → `dist/`
Browser SDK	Firebase Web SDK 12 (Auth, Firestore, Storage, Functions, App Check)
Backend	Firebase Functions **v1 API** on Node.js 22
Backend deps	`firebase-functions` ^7.3.2, `firebase-admin` ^13.10.0 (pinned to 13.x)
Tests	Vitest 4, Firebase Rules Unit Testing, Playwright 1.61, axe-core
Delivery	GitHub Actions + Firebase CLI pinned at `15.24.0` via `npx`

Node 22 is not optional — it matches the Functions runtime. The lockfiles, not the semver ranges in `package.json`, are the real bill of materials.

Note the version split: `functions/` pins `firebase-admin` to **13.x**, while the top-level dev dependency used by the `scripts/` tooling is **14.x**. Do not "align" these without testing Functions load.

4. Get it running

```
# Prerequisites: Node.js 22 LTS, Git, Firebase CLI, staging project access
Set-Location HYTech

npm ci                        # use ci, not install
npm ci --prefix functions
npx playwright install chromium

Copy-Item .env.staging.example .env.staging.local   # then fill in the values
npm run dev:staging
```

`VITE_`-prefixed variables are exposed to browser code. The Firebase web config values identify a project and are **not** secrets — security rests on Auth, Security Rules, App Check, and backend validation. The reCAPTCHA site key is likewise public.

Required: `VITE_FIREBASE_API_KEY`, `VITE_FIREBASE_AUTH_DOMAIN`, `VITE_FIREBASE_PROJECT_ID`, `VITE_FIREBASE_STORAGE_BUCKET`, `VITE_FIREBASE_MESSAGING_SENDER_ID`, `VITE_FIREBASE_APP_ID`, `VITE_RECAPTCHA_SITE_KEY`.

> `npm run dev` reads `.env.local` — **verify that file does not silently point at production.** Prefer `npm run dev:staging`.

Never commit `.env.local`, `.env.staging.local`, `.env.e2e.local`, service-account JSON, Firebase CI tokens, or real passwords.

Verify your setup

```
npm run build                     # production bundle must succeed
node --check functions/src/index.js
npm run test:unit
npm run test:rules                # boots the emulator
npm run test:e2e:public
```

Test accounts

There are no shared built-in credentials. Provision **staging** identities — active admin, active trainer, enrolled/unenrolled/pending/disabled students:

```
npm run qa:seed:identities
npm run qa:seed:data
npm run qa:verify-seed
```

Read each script and confirm its target project before running it.

5. Where the code lives

```
HYTech/
├── src/
│   ├── App.jsx                route table
│   ├── firebase.js            SDK init + App Check
│   ├── context/               AuthContext, ToastContext, hooks
│   ├── components/            admin, auth, dashboard, hytbot, landing, layout,
│   │                          logs, sectors, settings, shared, student,
│   │                          trainer, users
│   └── utils/firestoreService.js   the entire data-access layer
├── functions/src/index.js     ALL Cloud Functions
├── firestore.rules · storage.rules · firestore.indexes.json
├── scripts/                   QA seeding, provisioning, audit, cleanup, backup
└── tests/{unit,rules,e2e}/
```

Read these four files first, in this order

1. `firestore.rules` — the real authorization model
2. `functions/src/index.js` — everything trusted
3. `src/utils/firestoreService.js` — every client read/write
4. `src/App.jsx` — routes and guards

Maintainability hotspots

File	Lines	Why it matters
`components/trainer/ClassDetail.jsx`	7,712	Trainer class mgmt, builders, grading, gradebook
`utils/firestoreService.js`	5,149	Single data-access layer for the whole app
`components/student/StudentCourse.jsx`	3,981	Student class experience, quiz runner, submissions
`functions/src/index.js`	1,949	Every trigger and callable

These four carry most of the product behaviour and most of the risk. Splitting them is the top engineering priority; do it incrementally, behind tests.

6. Data model — essentials and traps

Top-level collections: `users`, `userSettings`, `sectors`, `courses`, `classes`, `classDirectory`, `enrollments`, `certificates`, `notifications`, `activityLogs`, `securityLogs`, `idRequests`, `incidentForms`, `config/appSettings`, `students/{uid}/progress/{classId}`.

```
classes/{classId}/
├── members/{studentId}                 duplicates enrollments
├── activity/{eventId}                  engagement telemetry, immutable
├── topics/{topicId}                    what the UI actually renders
├── modules/{moduleId}/materials/{id}   effectively dead
├── materials/{materialId}
├── announcements/{id}/comments/{id}
├── assessments/{assessmentId}          path A: assessment builder
│   ├── private/answerKey
│   └── attempts/{attemptId}
└── assignments/{assignmentId}          path B: form builder
    ├── private/answerKey
    ├── attempts/{attemptId}
    └── submissions/{studentId}
```

The five traps. Full ranked list of fifteen: docs/diagrams/as-built/database-schema.md.

1. `courses` **holds course *templates*, not classes.** `classes` holds real running classes — and `getCourses()` / `getCourseById()` read `classes`. This is the single most common source of wrong queries.
2. **Graded items live in two parallel collections.** Anything resolving one must check **both** `assessments` and `assignments`, and read attempts from the same parent that authored the item.
3. `status` **casing is split** — `'Active'` and `'active'` both occur. Rules compensate with `in ['Active','active']`; your queries will not.
4. **Progress and membership are each stored twice** and kept in step only by Cloud Functions.
5. `enrollments` **document IDs are composed:** `{classId}_{studentId}`. That composition is what enforces one enrolment per learner per class — Firestore has no unique constraints.

Enrolment states: `pending → active → ongoing → completed` (or `rejected`). Self-join creates `pending` and a trainer approves, unless an administrator turns `requireEnrollmentApproval` off in `config/appSettings`, in which case it goes straight to `active`.

Diagrams: docs/diagrams/ — design-level and as-built.

---

7. Security model — the non-negotiables

Five independent layers. Each re-checks; none trusts the one before it.

#	Layer	Enforces
1	Firebase Auth	Identity **only**
2	`users/{uid}`	Application role, account status, provisioning source
3	Route guards (`App.jsx`)	Navigation — **not security**
4	Firestore / Storage rules	Independent access control
5	Cloud Functions	Re-check auth, role, status, ownership, input

Route guards are not security. A hostile client can call the Firebase APIs directly. Every sensitive operation must be enforced in rules or in a validated Cloud Function.

Rules read the caller's own `users/{uid}` document via `canUseLms()`, which requires `status == 'Active'` **and** either a verified email, an admin/trainer role, or `createdBy == 'admin'`. That last clause waives email verification for admin-created accounts — which is exactly why the create rule forbids a self-registering user from ever setting `createdBy`. Do not relax it.

Server-write-only — no client write path exists

`assessments/*/attempts`, `assignments/*/attempts`, `students/{uid}/progress/{classId}`, `certificates`, `securityLogs`, `classDirectory`, and template answer keys.

Granting a browser write to any of these is a security regression, not a convenience fix — attempts and certificates are the anti-tampering boundary of the entire grading model.

Deletes are disabled in rules for sectors, courses, classes, and assessments. Deletion happens through the `delete*Secure` callables, which cascade so answer keys and attempts cannot be orphaned.

Other posture

- **Answer keys** live in a private child document, never in the trainee-readable item.
- **PII** (phone, address, birth date, emergency contact) lives in `users/{uid}/private/profile`, outside the directory-readable document.
- **Students may only notify staff** — this prevents a learner blasting notifications to every other learner.
- **Hosting headers:** HSTS (1 year, includeSubDomains, preload), `X-Frame-Options: DENY`, `X-Content-Type-Options`, Referrer-Policy, Permissions-Policy, COOP, and an **enforcing** CSP. `firebase.json` sets `Content-Security-Policy`, *not* `-Report-Only`, so a violation blocks the resource. Adding a new external origin requires a CSP change.
- **Avatars** are restricted to raster MIME types.

---

8. Assessments and grading — the genuinely tricky part

Two authoring paths produce quiz-like items. The assessment builder writes to `classes/{id}/assessments`; the form builder writes to `classes/{id}/assignments`. A `Submission`-type assignment collects uploaded work and is graded manually instead.

Grading is server-side and authoritative. `submitAssessmentAttempt` validates the caller, enrolment, publication state, availability window, attempt limit, and time limit; grades; then writes an **immutable** attempt inside a transaction. Browser clients cannot create or alter an attempt. The callable returns a `kind` field telling the client which collection now holds the attempt — read it back from there.

Paragraph answers cannot be auto-graded, so the attempt lands in `pending_review`; a trainer finalises it via `gradeAssessmentAttempt`.

Attempt document IDs are conditional: the trainee's `uid` when `settings.oneResponsePerUser` is set, otherwise an auto-ID. Never assume a stable attempt path.

Deadlines carry a time of day. Dates are authored with `datetime-local` and stored as absolute ISO instants. Legacy records hold a bare `YYYY-MM-DD`, which resolves to **end of day** for a deadline and **start of day** for an open date. Client and Cloud Function must apply this identically — if you change one, change both.

A missed deadline scores zero, but that zero is derived from "no attempt + deadline passed". No zero attempt document is ever written, so do not go looking for one. Tasks flagged `allowLateSubmissions` are exempt.

---

9. Cloud Functions

All in `functions/src/index.js`, region `asia-southeast1`, Functions **v1** API.

Triggers

Function	Fires on	Does
`syncClassDirectory`	`onWrite classes/{id}`	Rewrites the sanitised public mirror
`syncClassEnrollmentCount`	enrolment writes	Maintains `currentEnrollments`
`notifyTraineesOnAnnouncement`	new announcement	Fans out notifications
`notifyTraineesOnAssessmentPublish`	assessment publish	Fans out notifications
`notifyTraineesOnAssignmentPublish`	assignment publish	Fans out notifications

> `syncClassDirectory` writes with `merge: false`. Any field absent from `toClassDirectoryEntry()` is destroyed on every class write. Add new fields there or lose them.

Callables

Group	Functions
Grading & records	`submitAssessmentAttempt`, `gradeAssessmentAttempt`, `recalculateMyProgress`, `graduateEnrollment`, `revokeCertificate`, `verifyCertificate`
Administration	`adminUpdateUserAccount`, `changeEnrollmentStatus`, `promoteClassToTemplate`, `cloneTemplateToClass`
Cascading deletes	`deleteAssessmentSecure`, `deleteClassSecure`, `deleteCourseTemplateSecure`, `deleteSectorSecure`
Migrations	`migrateClassDirectory`, `migrateAssessmentAnswerKeys`

`verifyCertificate` is deliberately unauthenticated — it exists so a third party can validate a certificate number and token.

`graduateEnrollment` recalculates progress first and **refuses** unless every required item is satisfied; certificate issue and enrolment completion happen in one transaction.

Engineering rules

- Re-check auth, role, status, and ownership inside every callable. The client is not trusted.
- Use transactions or batches for multi-document consistency.
- Throw typed `HttpsError`s; the code shows up in the logs and the client branches on it.
- Confirm the client calls the `asia-southeast1` region, or you get "function not found".

---

10. Making a change

If you touch…	Also do
Firestore reads/writes	Update `firestore.rules`; add/adjust indexes; run `npm run test:rules`
A query with a new filter/sort	Add the composite index to `firestore.indexes.json` **and deploy it first**
Grading, attempts, progress, certificates	Change the Cloud Function, never the client; re-run academic tests
Assessment or assignment shape	Handle **both** collections; verify attempts still resolve
Uploads	Update `storage.rules`; check MIME and size limits
A new external origin (font, script, image, API)	Update the CSP in `firebase.json`
Roles or account status	Re-verify `canUseLms()` and every guard; never let a client set `role`, `status`, or `createdBy`
Anything user-visible	Check mobile/tablet/desktop, keyboard access, and labels

Ordering rule: deploy rules, indexes, and Functions **before** Hosting whenever the frontend depends on new backend behaviour.

Keep changes scoped and reversible. Never mix an unrelated dependency upgrade into a feature fix. Avoid browser-side writes for authoritative data.

Pull-request checklist

- [ ] Requirement and acceptance criteria stated
- [ ] Security impact reviewed
- [ ] Verified against **staging**, not production
- [ ] Production build passes; `node --check functions/src/index.js` passes
- [ ] Relevant unit, rules, and Playwright tests pass
- [ ] Mobile, tablet, desktop, keyboard, and labels checked for UI changes
- [ ] Functions, rules, and indexes included when the frontend needs them
- [ ] Migration and rollback documented
- [ ] No credentials, personal data, or internal audit files added

A change is done when rules and Functions support the behaviour, failure states are understandable, tests cover the risky path, staging verification passes, and rollback is possible — not when it merely works locally.

---

11. Testing

```
npm run test:unit             # Vitest
npm run test:rules            # rules against the emulator
npm run test:e2e:public       # public pages + accessibility
npm run test:e2e:responsive   # layout
npm run test:e2e:roles        # authenticated role journeys
npm run test:qa:full          # validate QA env, then all four browsers
npm run security:secrets      # secret scan
```

For authenticated tests, copy `.env.e2e.example` to `.env.e2e.local` with **staging-only** accounts. The Playwright config refuses arbitrary remote targets: it allows `hytech-lms-staging.web.app`, or localhost with `E2E_ALLOW_LOCAL=true`.

CI (`.github/workflows/qa.yml`) runs on every push and pull request: seeds staging fixtures, builds, then runs Chromium, Edge, Firefox, and WebKit. On failure it uploads a `playwright-evidence-<run>` artifact with the HTML report, traces, screenshots, and video (14-day retention). Open a trace with `npx playwright show-trace <path>`.

> CI retries a failing test twice, so **flakiness can hide behind a green check.** Treat a retried pass as a defect to investigate.

For assessment or class changes, also run the manual smoke flow in `docs/FULL_WEBSITE_QA_PLAYBOOK.md`.

12. Deploy and rollback

Production

QA gates deployment. `deploy.yml` is triggered by `workflow_run` on the **QA** workflow and runs only when QA concluded `success` on `main`. `workflow_dispatch` is the rollback escape hatch — it skips QA but is restricted to `main`. Both paths require the GitHub `production` environment approval.

The job then fails closed if `VITE_RECAPTCHA_SITE_KEY` is unset, runs `firebase deploy --dry-run` as validation, sets the Functions artifact cleanup policy, and finally runs a **bare** `firebase deploy` — no `--only`. That ships Hosting, Firestore rules and indexes, Storage rules, **and** Cloud Functions together. Assume every push to `main` that passes QA ships all of it.

Staging

```
npm run build:staging
npm run test:qa:full
firebase deploy --project staging --dry-run
firebase deploy --project staging
```

Rollback

- **Hosting:** use Hosting release history.
- **Functions / rules:** redeploy the last known-good commit.
- **Data:** there is **no automatic rollback for destructive Firestore mutations.** Use exports and tested migrations. `npm run backup:dump` exists; the restore path has not been exercised end-to-end (see §14).

13. Troubleshooting

Symptom	Most likely cause	Fix
"Firebase configuration is missing"	Wrong/absent `.env.*.local`, or dev server not restarted	Check the file exists in `HYTech/`, all `VITE_FIREBASE_*` are set, restart
"Missing or insufficient permissions"	`users/{uid}` role/status wrong, email unverified, or rules deployed to a *different* project	Verify the doc, then confirm rules were deployed to the project the frontend targets. **Do not weaken rules globally.**
Callable "not found"	Wrong region, not exported, or not deployed	Confirm `asia-southeast1`, export from `functions/src/index.js`, redeploy
Quiz submission fails	Unpublished, window closed, no active enrolment, or item in the *other* collection	Check publication + window + enrolment; check both collections; read the `HttpsError` code in the logs
Production UI unchanged	Build didn't run, wrong project, or backend not deployed	Confirm build, Hosting target, and that Functions/rules shipped too
New font/script/image silently blocked	Enforcing CSP	Add the origin to the CSP in `firebase.json`
Rules tests won't start	Emulator/Java or project mismatch	`npm run test:rules` uses `demo-hytech-lms`; check the emulator boots
Role E2E tests skip	`.env.e2e.local` missing or non-allowlisted target	Fill staging accounts; use the allowed host or `E2E_ALLOW_LOCAL=true`

14. Open risks and priorities

The dated authoritative list is `HYTech/docs/TURNOVER_RISK_REGISTER.md`. **Do not copy its claims into a release without re-checking the live projects.**

Review explicitly at handover: App Check enforcement state in production Firestore and Storage; live test or malformed accounts in production; historical credentials in Git history; Cloud Data Access audit-log coverage; unverified legacy academic progress; orphaned data after Auth deletion; no upload malware scanning; unexercised restore and unsigned RPO/RTO; long-lived CI credential instead of WIF/OIDC; operator workstation service-account keys; Storage soft-delete configuration; and outstanding privacy, academic, accessibility, incident, and retention sign-offs.

Engineering priorities, in order:

1. Close high-severity operational risks, with retained evidence.
2. Implement a governed user offboarding and data-cleanup path.
3. Exercise a full **non-production** restore.
4. Expand trusted academic workflow tests.
5. Replace the long-lived `FIREBASE_TOKEN` with workload identity federation.

6. Modularise `ClassDetail`, `StudentCourse`, `firestoreService`, and Functions.
7. Decide whether to expose, finish, or remove the dormant certificates UI.
8. Add content scanning before allowing untrusted public uploads.
9. Make linting and static analysis a stable CI gate.
10. Keep dependency risk dispositions owned and time-bound.

---

15. Handover acceptance checklist

Repository and engineering

- ☐ Repository access transferred; branch protection understood
- ☐ Receiving developer can install with **both** lockfiles
- ☐ Staging build, unit tests, rules tests, and Functions load all pass
- ☐ The four maintainability hotspots in §5 have been read
- ☐ No local secret or production export is present in the repository

Firebase and environments

- ☐ Production and staging project ownership assigned
- ☐ Auth, Firestore, Storage, Functions, Hosting, App Check access is least-privileged
- ☐ Region and billing responsibility understood
- ☐ App Check enforcement state recorded **from the live console**
- ☐ Firestore backup, PITR, delete protection, and Storage protection verified live

CI/CD and credentials

- ☐ GitHub `production` environment has named reviewers
- ☐ Staging QA accounts have owners and rotation/offboarding dates
- ☐ `FIREBASE_TOKEN` → WIF migration has a named owner
- ☐ No service-account keys on operator workstations

Operations and governance

- ☐ Restore procedure exercised in non-production, with evidence
- ☐ Incident, privacy, academic, accessibility, and retention sign-offs tracked
- ☐ Risk register re-verified against live projects on the handover date

---

16. Where to go deeper

Topic	Document
Everything, exhaustively	HYTECH_LMS_DEVELOPER_TURNOVER_GUIDE.md
Standing up a **new** Firebase environment	Long guide, §10 (~300 lines, step by step)
Dated operational risk record	`HYTech/docs/TURNOVER_RISK_REGISTER.md`
Security remediation operations	`HYTech/docs/SECURITY_REMEDIATION_OPERATIONS.md`
Dependency risk dispositions	`HYTech/docs/DEPENDENCY_RISK_REGISTER.md`
Environment/project separation	`docs/FIREBASE_ENVIRONMENTS.md`
Manual QA smoke flow	`docs/FULL_WEBSITE_QA_PLAYBOOK.md`
QA automation detail	`HYTech/docs/QA_AUTOMATION.md`
Schema, UML, use cases	docs/diagrams/
Short orientation	README.md

HYTech LMS Developer Turnover Brief · verified against the repository on 30 July 2026, branch `security/hardening-remediation`*. Live Firebase state can differ from the checked-out repository — verify the target project and the current risk register before any operational change.*